

Homework 3

Ravi Kini

November 24, 2025

Exercise 1

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ with $f(x_*) = f'(x_*) = 0$, $f''(x_*) \neq 0$ such that f''' is continuous on an interval around x_* . Since the continuity of f''' implies the continuity of f' , for any constant M , we can select $\delta_1 > 0$ such that for all x such that $|x - x_*| < \delta_1$, $|f(x) - f(x_*)| = |f(x)| < M$. We can also select $\delta_2 > 0$ such that x_* is the only root of $f(x)$ in $[x_* - \delta_2, x_* + \delta_2]$. It then follows that x_* is the only fixed point of:

$$T(x) = x - \frac{f(x)}{f'(x)}$$

in $[x_* - \delta_2, x_* + \delta_2]$. Letting $\delta = \min(\delta_1, \delta_2)$, by the Fixed-Point Theorem, for any $x_0 \in [x_* - \delta, x_* + \delta]$, the sequence where $x_{k+1} = T(x_k)$ converges to x_* . It then follows that Newton's method converges for x_0 sufficiently close to x_* , i.e. $x_0 \in [x_* - \delta, x_* + \delta]$.

We now let $f(x) = (x - x_*)^2 g(x)$, where $g(x_*) \neq 0$ and g is differentiable. Since x_* is the only root in the interval, that $g(x_k) \neq 0$ for $x_k \neq x_*$. Then:

$$f'(x) = 2(x - x_*)g(x) + (x - x_*)^2 g'(x)$$

Consequently, letting $e_k = |x_k - x_*|$, we see that:

$$\begin{aligned} e_{k+1} = |x_{k+1} - x_*| &= \left| x_k - \frac{f(x_k)}{f'(x_k)} - x_* \right| \\ &= \left| x_k - \frac{(x_k - x_*)^2 g(x_k)}{2(x_k - x_*)g(x_k) + (x_k - x_*)^2 g'(x_k)} - x_* \right| \\ &= \left| (x_k - x_*) - \frac{(x_k - x_*)g(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| \\ &= \left| 1 - \frac{g(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| e_k \\ &= \left| \frac{g(x_k) + (x_k - x_*)g'(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| e_k \\ \frac{e_{k+1}}{e_k} &= \left| \frac{g(x_k) + (x_k - x_*)g'(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| \end{aligned}$$

Taking the limit as $k \rightarrow \infty$, since $\lim_{k \rightarrow \infty} x_k - x_* = 0$:

$$\begin{aligned}
\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k} &= \lim_{k \rightarrow \infty} \left| \frac{g(x_k) + (x_k - x_*)g'(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| \\
&= \lim_{k \rightarrow \infty} \frac{1}{2} \left| \frac{2g(x_k) + 2(x_k - x_*)g'(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| \\
&= \lim_{k \rightarrow \infty} \frac{1}{2} \left| 1 + \frac{(x_k - x_*)g'(x_k)}{2g(x_k) + (x_k - x_*)g'(x_k)} \right| \\
&= \frac{1}{2}
\end{aligned}$$

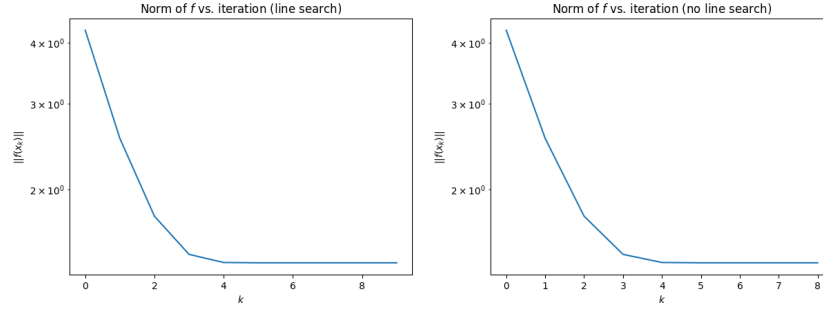
Exercise 2

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton(f, J, x0, tol_f=1e-6, tol_dx=1e-6, max_iter=100, line_search=True):
5     x = x0.copy()
6     history = np.zeros((max_iter, 3))
7     for k in range(max_iter):
8         fx = f(x)
9         Jx = J(x)
10
11         norm_f = np.linalg.norm(fx)
12         if norm_f < tol_f:
13             history[k] = (k, norm_f, 0.0)
14             return x, True, k, history
15
16         try:
17             dx = np.linalg.solve(Jx, -fx)
18         except:
19             return x, False, k, history
20         norm_dx = np.linalg.norm(dx)
21
22         alpha = 1
23         if line_search:
24             alpha = backtracking_line_search(f, x, fx, dx, norm_f)
25
26         x_new = x + alpha * dx
27         history[k] = (k, norm_f, norm_dx)
28         if norm_dx < tol_dx:
29             return x_new, True, k + 1, history
30         x = x_new.copy()
31     return x, False, max_iter, history
32
33 def backtracking_line_search(f, x, fx, dx, norm_f, rho=0.5, c=1e-4, max_iter=20):
34     alpha = 1
35     norm_f_sq = norm_f ** 2
36     phi_prime_0 = -norm_f_sq
37     for _ in range(max_iter):
38         x_new = x + alpha * dx
39         fx_new = f(x_new)
40         norm_f_new_sq = np.linalg.norm(fx_new) ** 2
41
42         if norm_f_new_sq <= norm_f_sq + c * alpha * phi_prime_0:
43             return alpha
44
45     alpha *= rho
46     return alpha
```

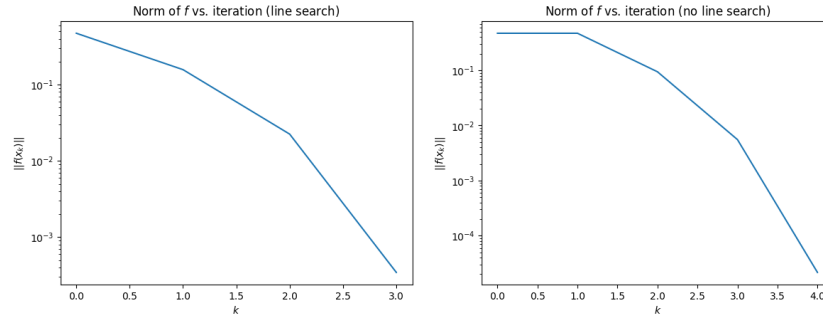
Part (a)

$$f(x_1, x_2) = \left(\frac{5 - 2x_1}{2x_2 - 3}, \frac{5 - 2x_2}{2x_1 - 3} \right)$$

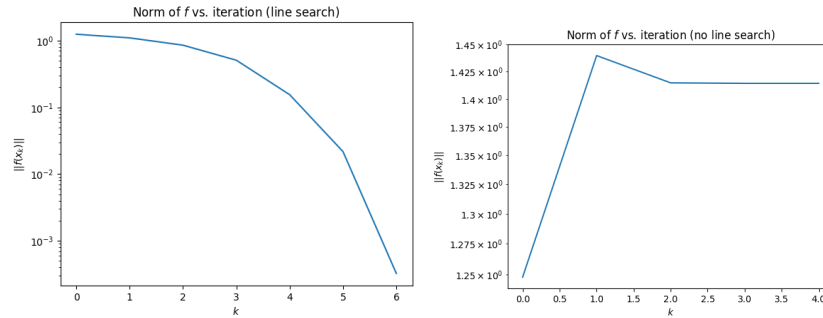
For $x_0 = (1, 1)$:



For $x_0 = (3, 3)$:



For $x_0 = (10, 10)$:

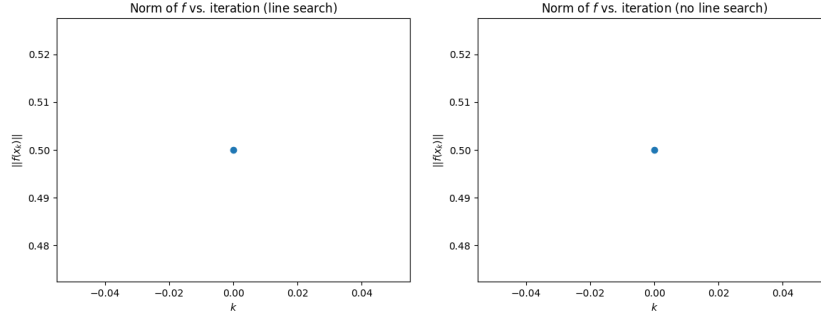


The system has singularities when $x_1 = 3/2$ or $x_2 = 3/2$, and the root is at $(x_1, x_2) = (5/2, 5/2)$. For an initial guess of $x_0 = (1, 1)$, which is closer to the singularity, Newton's method converges to a point close to $(0, 0)$ both with and without line search. For an initial guess of $x_0 = (3, 3)$, which is equally close to both the root and the singularity, Newton's method converges to the root, although it does so faster using line search. For an initial guess of $x_0 = (10, 10)$, which is far from both the root and the singularity, Newton's method converges to the root with line search and to a point close to $(0, 0)$ without line search. Line search is useful when a singularity is present, helping Newton's method to navigate away from the singularity.

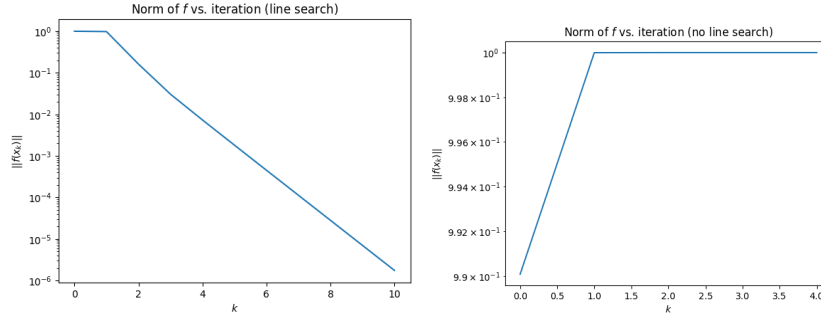
Part (b)

$$f(x) = \frac{x^2}{1+x^2}$$

For $x_0 = 1$:



For $x_0 = 10$:

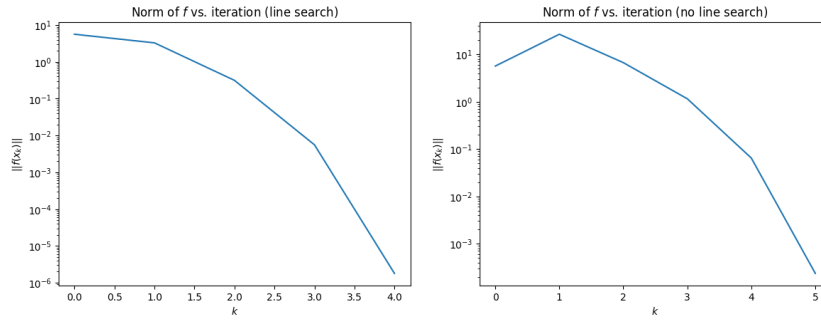


The system has no singularities, and the root is at $x = 0$. For large x , $f'(x) \approx 0$. For an initial guess of $x_0 = 1$, which is close to the root, Newton's method converges after one iteration, both with and without line search. For an initial guess of $x_0 = 10$, which is far from the root, Newton's method converges to the root with line search, and diverges without line search. Line search is useful when the function has a small gradient, resulting in very large steps.

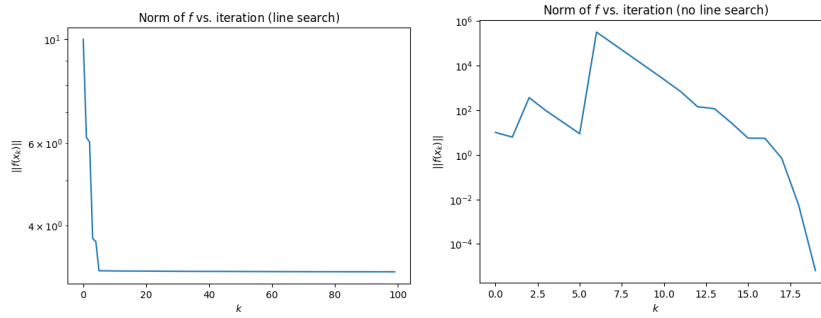
Part (c)

$$f(x_1, x_2) = (x_1x_2 + x_1^3 + 4, x_1x_2^2 + x_2 + 6)$$

For $x_0 = (-1, -1)$:



For $x_0 = (1, 1)$:

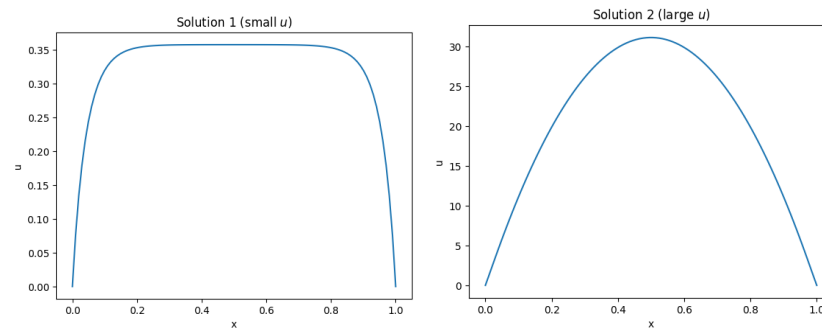


The system has no singularities, and the roots are at $(x_1, x_2) = (-1, 3)$ and $(x_1, x_2) \approx (-1.905, -1.531)$. For an initial guess of $x_0 = (-1, -1)$, which is close to both, Newton's method converges to a point close to the second root both with and without line search. For an initial guess of $x_0 = (1, 1)$, which is farther, Newton's method does not converge with line search, and converges to the first root without line search.

Exercise 3

Part (a)

```
1 eps = 1e-3
2 g = 0.25
3
4 N = 100
5 dx = 1 / (N + 1)
6 x = np.linspace(dx, 1 - dx, N)
7
8 coeff = eps / (dx ** 2)
9 A = np.zeros((N, N))
10
11 for i in range(N):
12     A[i, i] = -2 * coeff
13     if i > 0:
14         A[i, i-1] = coeff
15     if i < N - 1:
16         A[i, i+1] = coeff
17
18 def F(u):
19     return -np.exp(-np.abs(u)) * u + g
20
21 def dFdu(u):
22     result = np.zeros_like(u)
23     mask_pos = u > 0
24     mask_neg = u < 0
25     mask_zero = np.abs(u) < 1e-10
26
27     result[mask_pos] = -np.exp(-np.abs(u[mask_pos])) * (1 - u[mask_pos])
28     result[mask_neg] = -np.exp(-np.abs(u[mask_neg])) * (1 + u[mask_neg])
29     result[mask_zero] = -1.0
30     return result
31
32 def J(u):
33     return A + np.diag(dFdu(u))
34
35 def residual(u):
36     return np.dot(A, u) + F(u)
37
38 x_final = np.linspace(0, 1, N + 2)
39
40 u0 = np.ones(N) * g
41 u1, converged, iter, history = newton(residual, J, u0, line_search=True)
42 u1_final = np.zeros(N + 2)
43 u1_final[1:-1] = u1
44
45 u0 = -g * x * (1 - x) / (2 * eps)
46 u2, converged, iter, history = newton(residual, J, u0, line_search=True)
47 u2_final = np.zeros(N + 2)
48 u2_final[1:-1] = u2
49
50 plt.plot(x_final, u1_final)
51 plt.title('Solution 1 (small $u$)')
52 plt.xlabel('x')
53 plt.ylabel('u')
54 plt.show()
55
56 plt.plot(x_final, u2_final)
57 plt.title('Solution 2 (large $u$)')
58 plt.xlabel('x')
59 plt.ylabel('u')
60 plt.show()
```

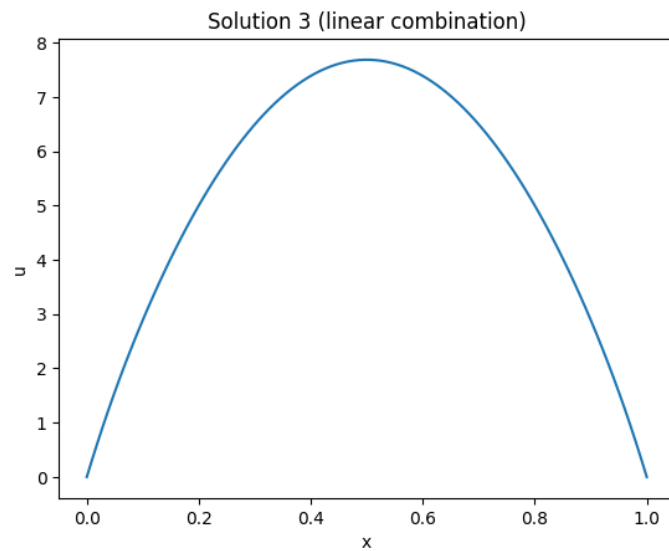


Part (b)

```

1 M = 1000
2 alpha_low = 0.1
3 alpha_high = 0.9
4 norms = np.zeros(M)
5 for a in range(M):
6     alpha = alpha_low + (alpha_high - alpha_low) * a / M
7     u0 = (1 - alpha) * u1 + alpha * u2
8     norms[a] = np.linalg.norm(residual(u0))
9     alpha = alpha_low + (alpha_high - alpha_low) * np.argmax(norms) / M
10    u3 = (1 - alpha) * u1 + alpha * u2
11    u3_final = np.zeros(N + 2)
12    u3_final[1:-1] = u3
13
14    plt.plot(x_final, u3_final)
15    plt.title('Solution 3 (linear combination)')
16    plt.xlabel('x')
17    plt.ylabel('u')
18    plt.show()

```



Exercise 4

Part (a)

```
1 m = 50
2 n = 12
3 x = np.linspace(0, 1, m)
4 A = np.vander(x, n)
5 b = np.cos(4 * x)
6
7 print(np.linalg.cond(A))
```

The condition number of A is approximately $\text{cond}(A) \approx 1.172 \cdot 10^8$.

Part (b)

```
1 def classical_GS(A):
2     m, n = A.shape
3     Q = np.zeros((m, n))
4     R = np.zeros((n, n))
5
6     for j in range(n):
7         v = A[:, j].copy()
8         for i in range(j):
9             R[i, j] = np.dot(Q[:, i], v)
10            v -= R[i, j] * Q[:, i]
11        R[j, j] = np.linalg.norm(v)
12        if R[j, j] > 1e-14:
13            Q[:, j] = v / R[j, j]
14        else:
15            Q[:, j] = v
16        return Q, R
17
18 def modified_GS(A):
19     m, n = A.shape
20     Q = A.copy()
21     R = np.zeros((n, n))
22
23     for j in range(n):
24         R[j, j] = np.linalg.norm(Q[:, j])
25         if R[j, j] > 1e-14:
26             Q[:, j] /= R[j, j]
27         for k in range(j + 1, n):
28             R[j, k] = np.dot(Q[:, j], Q[:, k])
29             Q[:, k] -= R[j, k] * Q[:, j]
30        return Q, R
31
32 def householder(A):
33     m, n = A.shape
34     Q = np.eye(m)
35     R = A.copy()
36
37     for j in range(n):
38         x = R[j:, j]
39         e1 = np.zeros_like(x)
40         e1[0] = 1
41         v = x + np.sign(x[0]) * np.linalg.norm(x) * e1
42         v_norm = np.linalg.norm(v)
43         if v_norm > 1e-14:
44             v /= v_norm
45             R[j:, j:] -= 2 * np.outer(v, np.dot(v, R[j:, j:]))
```

```

46     Q[:, j:] -= 2* np.outer(np.dot(Q[:, j:], v), v)
47     return Q[:, :n], R[:, n, :]

```

Since Q should be orthogonal, $Q^T Q - I = 0$. For the classical Gram-Schmidt method, the computed $\|Q^T Q - I\|_2 \approx 4.881 \cdot 10^{-9}$. For the modified Gram-Schmidt method, the computed $\|Q^T Q - I\|_2 \approx 4.881 \cdot 10^{-9}$. For the Householder method, the computed $\|Q^T Q - I\|_2 \approx 1.486 \cdot 10^{-15}$. The matrix computed using the Householder method has the lowest norm since it is backwards stable.

Part (c)

```

1  x = np.linalg.solve(np.matmul(np.transpose(A), A), np.matmul(np.transpose(A), b))
2  print(np.linalg.norm(x - a))
3  print(np.linalg.norm(np.matmul(A, x) - b))
4
5  Q, R = classical_GS(A)
6  x = np.linalg.solve(R, np.matmul(np.transpose(Q), b))
7  print(np.linalg.norm(x - a))
8  print(np.linalg.norm(np.matmul(A, x) - b))
9
10 Q, R = modified_GS(A)
11 x = np.linalg.solve(R, np.matmul(np.transpose(Q), b))
12 print(np.linalg.norm(x - a))
13 print(np.linalg.norm(np.matmul(A, x) - b))
14
15 Q, R = householder(A)
16 x = np.linalg.solve(R, np.matmul(np.transpose(Q), b))
17 print(np.linalg.norm(x - a))
18 print(np.linalg.norm(np.matmul(A, x) - b))
19
20 x = np.linalg.lstsq(A, b)[0]
21 print(np.linalg.norm(x - a))
22 print(np.linalg.norm(np.matmul(A, x) - b))

```

Method	$\ x - a\ _2$	$\ Ax - b\ _2$
Normal equations	$1.654 \cdot 10^0$	$1.287 \cdot 10^{-7}$
Classical GS	$1.957 \cdot 10^{-1}$	$1.795 \cdot 10^{-8}$
Modified GS	$1.957 \cdot 10^{-1}$	$1.795 \cdot 10^{-8}$
Householder	$1.611 \cdot 10^{-9}$	$7.999 \cdot 10^{-9}$
Built-in solver	$1.437 \cdot 10^{-8}$	$7.999 \cdot 10^{-9}$

The normal equations method has the worst performance, as expected since the condition number of the matrix is high, making it ill-conditioned. The classical and modified Gram-Schmidt perform comparably, with some error due to loss of orthogonality when computing Q . The Householder and built-in least squares solver perform the best, as expected since they are backward stable.